

RunPod 스토리지 접속 & GPU 학습

네트워크 볼륨 기반 GPU Pod 워크플로우 · 사용법 리포트

대상 스크립트 `runpod_test/runpod_storage_test.py` | 작성일 2026-07-22 | 공식 `runpod` Python SDK 사용 (S3 불필요)

1. 개요

팀 공용 RunPod 네트워크 볼륨(스토리지)에 파이썬 SDK로 접속해 GPU Pod을 띄우고, Pod에 접속해 코드(git)와 데이터(구글드라이브)를 넣어 학습을 돌리기까지의 전체 과정을 정리한 리포트입니다. 별도의 S3 설정 없이 `pip install runpod` 하나로 동작합니다.

주의 여기서 '스토리지 접속'은 로컬 PC에서 볼륨 안의 파일을 직접 여는 것이 아니라, 볼륨을 **확인**하고 GPU Pod에 **마운트**하는 것을 뜻합니다. 실제 파일 작업(데이터 다운로드·학습)은 Pod 안에서 이루어집니다.

전체 흐름

- 0단계 — PowerShell을 열고 스크립트가 있는 폴더로 이동 (`cd`)
- 1단계 — SDK로 계정 접속 → 네트워크 볼륨 목록·데이터센터 확인 (`volumes / gpus`)
- 2단계 — GPU를 골라 볼륨을 붙인 Pod 생성 (`create`) → 볼륨이 `/workspace`에 마운트
- 3단계 — `status`로 주소 확인 후 브라우저로 **Jupyter Lab 접속** (비밀번호 `runpod-test`)
- 4단계 — Jupyter 터미널에서 코드를 `git clone` → `/workspace`
- 5단계 — 데이터를 구글드라이브에서 `gdown`으로 `/workspace`에 다운로드
- 6단계 — GPU 확인 후 학습 → 끝나면 정리 (`stop / terminate`, 볼륨 데이터는 유지)

2. 사전 준비

- **패키지 설치** — 로컬 PC에서 `pip install runpod` (구글드라이브 다운로드용 `gdown`은 Pod 안에서 설치합니다.)
- **API 키** — 스크립트 폴더의 `.env`에 `RUNPOD_API_KEY=본인키` 를 넣습니다. 팀 계정이면 팀장 키를 사용.
발급: `console.runpod.io` → Settings → API Keys
- **네트워크 볼륨** — RunPod 콘솔 → Storage → New Network Volume 에서 미리 생성해 둡니다.
- **크레딧** — Billing 메뉴에서 충전 (GPU는 시간당 과금).

3. 명령어 요약

명령	역할
<code>volumes</code>	내 네트워크 볼륨(스토리지) 목록과 각 볼륨의 데이터센터 확인
<code>gpus</code>	<code>create</code> 에 넣을 GPU id 목록 (가격까지 보려면 <code>runpod_test.py gpus</code>)
<code>create</code>	지정한 볼륨을 붙여 GPU Pod 생성 — 이 순간부터 과금 시작

명령	역할
status	Pod 상태와 접속 정보(Jupyter 주소·비밀번호) 확인
stop	Pod 정지 — GPU 반납(과금 멈춤), 볼륨·디스크는 유지
terminate	Pod 완전 삭제 — 과금 종료, 볼륨 데이터는 그대로 유지

4. 단계별 사용법

주의 이 문서에서 <VOLUME_ID>, <POD_ID>, <파일ID>처럼 꺾쇠로 쓴 부분은 **자리표시자**입니다. **예시를 그대로 치지 말고**, 꺾쇠(< >)까지 지운 자리에 명령을 실행해 나온 **본인 값**을 넣으세요.

0단계 · 스크립트 폴더로 이동

아래 명령들은 모두 `runpod_storage_test.py`가 있는 `runpod_test` 폴더에서 실행합니다.
PowerShell을 열고 저장소 폴더로 간 뒤, 그 안의 `runpod_test`로 이동하세요.

```
cd <hn_old-building 저장소를 받은 경로> # 사람마다 다름
cd runpod_test # 저장소 안의 스크립트 폴더
```

주의 첫 줄은 **본인이 저장소를 받은 위치**라 사람마다 다릅니다 (탐색기에서 저장소 폴더 주소를 복사해 붙여넣으면 됩니다). 이미 저장소 폴더에서 터미널을 열었다면 `cd runpod_test` 한 줄이면 되고, 폴더 안에 스크립트가 있는지는 `ls`로 확인할 수 있습니다.

1단계 · 볼륨(스토리지) 확인

계정에 접속해 네트워크 볼륨과 각 볼륨이 위치한 데이터센터를 확인합니다.

```
python runpod_storage_test.py volumes
```

출력 예시:

```
VOLUME_ID (--volume-id 로 사용)  이름      크기  데이터센터
-----
<VOLUME_ID>                     <볼륨이름>  20GB  EUR-IS-1
↑ 실제 실행하면 이 자리에 본인 볼륨의 진짜 id가 나옵니다
```

여기서 나온 **VOLUME_ID**와 **데이터센터**를 확인해 둡니다. GPU는 이 볼륨과 같은 데이터센터의 재고에서 배정됩니다. 고를 수 있는 GPU id는 다음으로 확인합니다.

```
python runpod_storage_test.py gpus
```

주의 GPU id는 목록에 나온 값과 글자까지 똑같이 넣어야 합니다. 예를 들어 A5000은 GeForce가 아니라 NVIDIA RTX A5000입니다('GeForce' 없음). 틀리면 No GPU found with the specified ID 에러가 나지만, 이는 배포 전 검증 단계라 **과금은 전혀 없습니다**. 목록 값을 그대로 복사해 쓰세요.

2단계 · 볼륨을 붙인 GPU Pod 생성

1단계에서 확인한 볼륨 id와 원하는 GPU id를 넣어 Pod을 생성합니다. 볼륨은 /workspace에 마운트됩니다.

```
python runpod_storage_test.py create --volume-id <VOLUME_ID> --gpu "NVIDIA RTX A4000"
```

<VOLUME_ID> 자리에는 1단계 volumes 출력에 나온 **본인 볼륨의 실제 id**를 넣습니다.

create 옵션

옵션	설명
--volume-id	(필수) 붙일 네트워크 볼륨 id — volumes 명령으로 확인
--gpu	사용할 GPU id (기본: NVIDIA GeForce RTX 3090)
--name	Pod 이름 (기본: storage-train-pod)
--image	도커 이미지 (기본: runpod/pytorch 2.4.0 · cuda12.4)
--disk	컨테이너 디스크 GB — 볼륨과 별개 (기본: 20)
--mount	볼륨 마운트 경로 (기본: /workspace)

주의 네트워크 볼륨은 **Secure Cloud 전용**이라 자동으로 SECURE로 생성됩니다. **create가 완료되는 순간부터 과금이 시작**됩니다. 해당 데이터센터에 그 GPU 재고가 없으면 생성이 실패하는데, 그럴 땐 다른 GPU id로 다시 시도하면 됩니다. 성공하면 **POD_ID**가 출력됩니다.

3단계 · Pod 접속 (Jupyter Lab)

생성이 끝나면 접속 정보를 확인합니다. 상태가 RUNNING이어도 컨테이너 안에서 Jupyter가 완전히 뜨고 프록시가 연결되기까지 **30초~2분** 더 걸립니다. 그동안은 접속이 안 되는 것처럼 보이다가 잠시 뒤 열리니, 안 되면 status를 몇 번 다시 실행하세요.

```
python runpod_storage_test.py status <POD_ID>
```

출력된 **Jupyter Lab** 주소를 브라우저로 열고, 로그인 화면의 **Password** 칸에 runpod-test를 입력하면 들어갑니다.

```
https://<POD_ID>-8888.proxy.runpod.net # Jupyter Lab 주소
Password: runpod-test # 로그인 비밀번호
```

4단계 · 코드 가져오기 (git clone)

Jupyter의 터미널을 열고, 볼륨 폴더(/workspace)에서 팀 저장소를 clone 합니다. 공개 저장소라 인증 없이 바로 받아집니다.

```
cd /workspace
git clone https://github.com/gpwhdqkr/hn_old-building.git
cd hn_old-building
ls
```

- 이미 받아둔 경우(팀원이 먼저 clone) → `cd /workspace/hn_old-building && git pull` 로 최신화.
- 필요 패키지는 `pip install -r requirements.txt` (있을 때). `torch-cuda`는 이미지에 이미 포함되어 있습니다.
- 비공개 저장소라면 clone 시 GitHub 개인 토큰이 필요합니다.

주의 /workspace는 볼륨이라 한 번 clone 해두면 Pod을 지워도 코드가 남습니다. 다음 Pod에서는 clone 대신 `git pull`만 하면 최신 상태가 됩니다.

5단계 · 데이터 가져오기 (구글드라이브 → gdown)

데이터는 구글드라이브에서 gdown으로 받습니다. **공유링크 전체가 아니라 링크 속 ID만** 넣습니다 (Pod에 설치된 gdown 버전에 따라 링크--fuzzy 방식은 없는 경우가 있어, ID 방식이 버전과 무관하게 동작합니다). 받는 위치는 /workspace(또는 repo의 data 폴더)여야 볼륨에 영구 보존됩니다.

```
pip install gdown
cd /workspace/hn_old-building/data      # 코드가 기대하는 데이터 경로로 이동
gdown <파일ID>                          # 파일 하나
gdown --folder <폴더ID>                 # 폴더째
```

공유링크에서 ID 찾는 위치:

파일: <https://drive.google.com/file/d/<파일ID>/view?usp=sharing>
폴더: <https://drive.google.com/drive/folders/<폴더ID>?usp=sharing>

주의 구글드라이브 파일/폴더 권한을 반드시 '**링크가 있는 모든 사용자**'로 바꾸세요. 안 그러면 gdown이 파일 대신 **HTML 권한 페이지**를 받아 실패합니다(다운로드가 몇 KB로 끝나면 이 문제입니다).

- 폴더에 파일이 **50개를 넘으면** --folder가 50개에서 잘립니다. 데이터셋은 구글드라이브에서 **zip으로 묶어** 하나로 올린 뒤 zip의 파일ID로 gdown 받아 unzip 하는 게 빠르고 안전합니다. (--folder가 없다고 나오는 구버전 gdown이면 `pip install -U gdown` 후 재시도)

6단계 · 작업 시작 & 정리 (과금 종료)

코드와 데이터가 준비되면, 먼저 GPU가 잡히는지 확인하고 학습을 시작합니다.

```
python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"
```

→ True NVIDIA RTX A4000 처럼 나오면 GPU 정상입니다. 이후 학습 스크립트를 실행하고, 작업이 끝나면 반드시 Pod을 내려 과금을 멈춥니다.

```
python runpod_storage_test.py stop <POD_ID>      # 정지: GPU 반납, 볼륨·디스크 유지
```

```
python runpod_storage_test.py terminate <POD_ID>  # 삭제: 과금 종료, 볼륨 데이터는 유지
```

주의 학습 결과(모델·체크포인트)는 반드시 `/workspace` 안에 저장하세요. `/workspace` 밖의 경로나 `pip install`한 패키지는 **terminate** 하면 모두 사라집니다. 볼륨의 코드·데이터·결과만 남습니다.

5. 주의사항 & 팁

- **과금은 create부터** — 생성 완료 시점부터 시간당 과금됩니다. 브라우저 탭을 닫아도 Pod은 계속 돌아가며 과금돼요. 끝나면 반드시 `terminate.runpod_test.py list`로 남은 Pod 확인 습관을 들이세요.
- **/workspace만 영구** — 볼륨(`/workspace`)의 코드·데이터·결과만 `terminate` 후에도 남습니다. 그 밖의 경로와 `pip install`한 패키지는 새 Pod에서 다시 준비해야 합니다.
- **GPU id 정확히** — `--gpu` 값은 `gpus` 목록의 id와 글자까지 일치해야 합니다 (예: NVIDIA RTX A5000, 'GeForce' 아님).
- **접속 타이밍** — 상태가 RUNNING이어도 Jupyter가 열리기까지 30초~2분 걸립니다. 안 되면 잠시 뒤 다시 시도하세요.
- **GPU 재고·데이터센터** — 네트워크 볼륨은 특정 데이터센터에 묶여, 그 데이터센터에 재고가 있는 GPU만 배정됩니다. 없으면 다른 GPU로 재시도.
- **키 보안** — API 키가 담긴 `.env`는 `.gitignore`에 있어 커밋되지 않게 유지하세요.